```
end
```

The default error message for this helper is *"must be accepted"*.

It can receive an `:accept` option, which determines the value that will be considered acceptance. It defaults to "1" and can be easily changed.

```
class Person < ActiveRecord::Base
  validates :terms_of_service, acceptance: { accept: 'yes' }
end
```

2.2 validates_associated

You should use this helper when your model has associations with other models and they also need to be validated. When you try to save your object, `valid?` will be called upon each one of the associated objects.

```
class Library < ActiveRecord::Base
  has_many :books
  validates_associated :books
end
```

This validation will work with all of the association types.

Don't use `validates_associated` on both ends of your associations. They would call each other in an infinite loop.

The default error message for `validates_associated` is *"is invalid"*. Note that each associated object will contain its own `errors` collection; errors do not bubble up to the calling model.

2.3 confirmation

You should use this helper when you have two text fields that should receive exactly the same content. For example, you may want to confirm an email address or a password. This validation creates a virtual attribute whose name is the name of the field that has to be confirmed with "_confirmation" appended.

```
class Person < ActiveRecord::Base
  validates :email, confirmation: true
end
```

In your view template you could use something like

```
<%= text_field :person, :email %>
<%= text_field :person, :email_confirmation %>
```

This check is performed only if `email_confirmation` is not `nil`. To require confirmation, make sure to add a presence check for the confirmation attribute (we'll take a look at `presence` later on this guide):

```
class Person < ActiveRecord::Base
  validates :email, confirmation: true
  validates :email_confirmation, presence: true
end
```

The default error message for this helper is *"doesn't match confirmation"*.

2.4 exclusion

This helper validates that the attributes' values are not included in a given set. In fact, this set can be any enumerable object.

```
class Account < ActiveRecord::Base
  validates :subdomain, exclusion: { in: %w(www us ca jp),
    message: "%{value} is reserved." }
end
```